Laporan Proyek Berpikir Komputasional (WI1102)

Kelas K05



Disusun Oleh:

Jovan Nathaniel Permana (16026295)
Qamila Annur Firdausi (16026380)
Abira Mutia Salsabila (16026060)
Ezra Hadyan Wicaksono (16026055)

Fakultas FMIPA Institut Teknologi Bandung

Semester 1 Tahun 2026/2027

1 Dasar-dasar Berpikir Komputasional yang Digunakan

1.1 Algoritma Newton-Raphson

1.1.1 Dekomposisi

Algoritma Newton-Raphson sebenarnya adalah algoritma optimasi yang bersifat iteratif, dengan tujuan utama untuk mencari akar-akar/titik potong sebuah fungsi $f(x)$ dengan sumbu x. Dalam proyek ini, algoritma Newton-Raphson difokuskan untuk mencari akar-akar dari sebuah polinom dengan derajat tertentu, sesuai masukan dari pengguna (*user*). Melalui prinsip dekomposisi (pemecahan suatu masalah dengan skala besar menjadi skala yang lebih kecil), algoritma Newton-Raphson untuk mencari akar-akar sebuah fungsi polinom dapat dipecah menjadi beberapa tugas berikut.

1. Menginput derajat polinom (N), nilai koefisien-koefisiennya, serta nilai tebakan awal (x_0) dari user.
2. Untuk nilai tebakan awal $x = x_0$ yang diberikan, hitung nilai fungsi $f(x_0)$ beserta turunannya, yaitu $f'(x_0)$.
3. Nilai x diperbarui secara terus-menerus melalui prosedur iteratif dengan persamaan
$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)} \tag{1}$$
hingga nilai *error* yang terjadi dalam penentuan akar menjadi tidak signifikan atau berada di bawah ambang batas tertentu.
4. Jika $f'(x) = 0$ pada suatu tahap tertentu atau perulangan pada langkah 3 tidak dapat selesai (*infinite loop*), maka proses perhitungan akar gagal dilakukan.

1.1.2 Pengenalan Pola

Algoritma Newton-Raphson memiliki pola yang mirip dengan algoritma optimasi, yaitu algoritma yang menekankan perhitungan matematika secara iteratif (berulang) untuk mencapai suatu tujuan. Dalam algoritma Newton-Raphson, persamaan (1) dijalankan secara berulang-ulang untuk mencapai suatu tujuan, yaitu akar persamaan polinom. Selama perulangan tersebut, nilai x akan terus diperbarui (*update*) hingga error dalam pencarian akar menjadi tidak signifikan atau lebih kecil dari ambang batas tertentu.

1.1.3 Abstraksi

Algoritma Newton-Raphson membutuhkan beberapa informasi penting, yaitu:

1. Derajat polinom N.
2. Koefisien-koefisien pada polinom.
3. Nilai tebakan awal x_0.

Dalam proyek ini, derajat polinom N disimpan di dalam variabel bertipe `int`, sementara nilai tebakan awal x_0 disimpan di dalam variabel bertipe `float`. Nilai-nilai koefisien pada polinom disimpan di dalam sebuah array `float` yang berukuran $N + 1$.

1.2 Algoritma Pinjam Buku

1.2.1 Dekomposisi

Algoritma pinjam buku merupakan algoritma berbasis teori graf (*graph theory*). Hubungan antar peminjam buku dapat direpresentasikan sebagai graf, dimana *nodes*/titik dari graf tersebut adalah peminjam dan *edges*/garis hubungan dari graf tersebut adalah hubungan antar peminjam. Graf yang dihasilkan adalah graf siklik (*cyclic graph*, dimana seseorang dapat bergerak dari satu *node* ke *node* lain, lalu kembali memutar ke titik semula) dengan *edge* dua arah. Selain itu, faktor umur yang diperhitungkan dalam algoritma pinjam buku (untuk menentukan peminjam berikutnya berdasarkan umur terendah) membuat graf ini menjadi graf berbobot (*weighted graph*). Oleh karena itu, dalam algoritma ini, digunakan algoritma DFS (*Depth First Search*) yang disertai dengan *backtracking*, dimana algoritma pinjam buku dapat dibagi menjadi beberapa tugas besar berikut.

1. Menginput matriks ketetanggaan (yang menggambarkan hubungan antar peminjam), umur setiap peminjam, dan peminjam pertama yang mendapatkan buku.
2. Selanjutnya, melalui matriks ketetanggaan, hubungan antara satu peminjam dengan peminjam lain dapat ditentukan. Apabila terdapat lebih dari satu peminjam, pilih peminjam dengan umur terendah dan prioritaskan peminjam yang belum pernah meminjam buku.

3. Jika buku sudah sampai di *node* buntu atau semua peminjam yang berhubungan dengan *node* tersebut sudah pernah meminjam buku sebelumnya, lakukan *backtracking*, yaitu mengembalikan buku ke peminjam sebelumnya.

4. Langkah 2 sampai 3 dilakukan terus-menerus sampai semua peminjam sudah pernah meminjam buku dan buku dikembalikan ke peminjam awal.

1.2.2 Pengenalan Pola

1.2.3 Abstraksi

1.3 Algoritma Labirin

1.3.1 Dekomposisi

1.3.2 Pengenalan Pola

1.3.3 Abstraksi

2 Lampiran

2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram

2.1.1 Algoritma Newton-Raphson

Kode 1: Kode Python algoritma Newton-Raphson tanpa subprogram

```
1  import time
2  """ Newton Raphson (tanpa subprogram) """
3
4  # program utama
5  # input
6  N = int(input('Masukkan derajat polinomial: '))
7  N += 1
8  coefs = [0]*N
9  for i in range(N):
10     if(i == 0):
11         val = float(input('Masukkan nilai konstanta: '))
12     else:
13         val = float(input(f'Masukkan nilai koefisien x^{
               i}: '))
14
```

```
    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = 0.0
    el = 0
    element = 1
    while el <= (N-1):
        val += coefs[el] * (x ** el)
        el += 1
    grad = 0.0
    while element <= (N-1):
        grad += element * coefs[element] * (x ** (
            element - 1))
        element += 1

    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f}
        Gradien: {grad:.6f}')
    delta = x_new - x
    if delta < 0:
        delta = -delta

    if val > 0:
        absval = val
    else:
        absval = -val

    if (delta < epsilon) and (absval < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
```

```
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.1.2 Algoritma Pinjam Buku

Kode 2: Kode Python algoritma Pinjam Buku tanpa subprogram

```
import time
"""Pinjam Buku (Tanpa Subprogram)"""
# program utama
# input
N = int(input("Masukkan jumlah orang: "))
matrix = []
umur = [0 for _ in range(N)]
sudah_pinjam = [0 for _ in range(N)]
riwayat_pinjam = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Masukkan isi matrix ketetanggaan untuk koordinat ({x},{y}): '))
        matrix[y][x] = val
    print('================================')

for i in range(N):
    val_umur = int(input(f'Masukkan umur untuk index {i}: '))
    umur[i] = val_umur

index_pinjam = int(input('Masukkan index tempat awal buku berada: '))
awal = time.perf_counter()
sudah_pinjam[index_pinjam] = 1
```

```
riwayat_pinjam += [index_pinjam]
top = 0
print('Urutan index peminjaman buku: ', end='')
print(index_pinjam, end='')
while True:
    koneksi = matrix[index_pinjam]
    # pindah peminjam
    i = 0
    kandidat = -1
    umur_min = 999999
    while i < N:
        # cek koneksi dan status pinjam
        if (koneksi[i] == 1) and (sudah_pinjam[i] == 0):
            # Cek umur termuda
            if umur[i] < umur_min:
                umur_min = umur[i]
                kandidat = i
        i += 1

    if kandidat != -1:
        index_pinjam = kandidat
        sudah_pinjam[index_pinjam] = 1
        riwayat_pinjam += [index_pinjam]
        top += 1
        print(" ->", index_pinjam, end="")
    else:
        # Pengecekan apakah semua orang sudah pernah meminjam
        jumlah = 0
        for items in sudah_pinjam:
            jumlah += items
        # selesai hanya jika semua orang sudah pernah meminjam dan buku kembali ke peminjam asal
        if top == 0 and jumlah == N:
            break
        # Jika belum selesai tapi mentok, backtrack ke orang sebelumnya
        top -= 1
        index_pinjam = riwayat_pinjam[top]
        print(" ->", index_pinjam, end="")

akhir = time.perf_counter()
eks = akhir - awal
```

```
print(f'\n\nWaktu eksekusi: {eks} detik.')
```

2.1.3 Algoritma Labirin

Kode 3: Kode Python algoritma Labirin tanpa subprogram

```
import time
"""labirin (tanpa Subprogram)"""
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Apakah koordinat ({x},{y})
            bisa dilewati (1) atau tidak (0): '))
        matrix[y][x] = val
    print('================================')

# proses
start_time = time.perf_counter()
# inisialisasi pointer dan arahnya
status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
# cari titik awal di sisi atas
for i in range(N):
    if matrix[0][i] == 1:
        status_sisi[0] = [i,0]
        break
# cari titik awal di sisi bawah
for i in range(N):
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
```

```
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
```

```
76          direction = [1,1,1,1]
77
78      # cek apakah bisa dilewati
79      i = 0
80      while(i < 4):
81          if direction[i] == 1:
82              if (i == 0) and (forbidden_move[i] == 0):
83                  bukan_dinding = int(matrix[pointer
                        [1]-1][pointer[0]] == 1)
84                  avail_move[i] = bukan_dinding
85              elif (i == 1) and (forbidden_move[i] == 0):
86                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]+1] == 1)
87                  avail_move[i] = bukan_dinding
88              elif (i == 2) and (forbidden_move[i] == 0):
89                  bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
90                  avail_move[i] = bukan_dinding
91              elif (i == 3) and (forbidden_move[i] == 0):
92                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]-1] == 1)
93                  avail_move[i] = bukan_dinding
94          else:
95              avail_move[i] = 0
96          i += 1
97      # Jika jalan buntu
98      if avail_move == [0,0,0,0]:
99          i = 0
100         while(i < 4):
101             if direction[i] == 1:
102                 if i == 0:
103                     bukan_dinding = int(matrix[pointer
                            [1]-1][pointer[0]] == 1)
104                     avail_move[i] = bukan_dinding
105                 elif i == 1:
106                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]+1] == 1)
107                     avail_move[i] = bukan_dinding
108                 elif i == 2:
109                     bukan_dinding = int(matrix[pointer
                            [1]+1][pointer[0]] == 1)
110                     avail_move[i] = bukan_dinding
111                 else:
```

```
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]-1] == 1)
                    avail_move[i] = bukan_dinding
            else:
                avail_move[i] = 0
            i += 1

    # Pilih jalur
    if clockwise:
        j = 0
        while (j < 4):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j += 1
    else:
        j = 3
        while (j >= 0):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
```

```
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j -= 1

    if (change_clockwise % (4*N)) == 0:
        clockwise = not clockwise

print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
   dilewati')
end_time = time.perf_counter()
execution_time = end_time - start_time
print('Labirin Diselesaikan!')
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram

2.2.1 Algoritma Newton-Raphson

Kode 4: Kode Python algoritma Newton-Raphson dengan subprogram

```
import time
""" Newton Raphson (dengan subprogram) """

# definisi fungsi
def function(x, coefs):
    value = 0.0
    for i in range(len(coefs)):
        value += coefs[i] * (x**i)
    return value

def gradient(x, coefs):
    value = 0.0
    for i in range(1, len(coefs)):
        value += i * coefs[i] * (x**(i - 1))
    return value

def absol(x):
    value = (x >= 0) * x + (x < 0) * (-x)
```

```
    return value

# program utama
# input
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')

    if (absol(x_new - x) < epsilon) and (absol(val) < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break
```

```
58
59      x = x_new
60
61  end_time = time.perf_counter()
62  execution_time = end_time - start_time
63  print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2.2 Algoritma Pinjam Buku

Kode 5: Kode Python algoritma Pinjam Buku dengan subprogram

```
1   import time
2   """Pinjam Buku (dengan Subprogram)"""
3   # definisi fungsi
4   def cari_kandidat(koneksi, umur, sudah_pinjam, N):
5       # pindah peminjam
6       i = 0
7       kandidat = -1
8       umur_min = 999999
9       while i < N:
10          # cek koneksi dan status pinjam
11          if (koneksi[i] == 1) and (sudah_pinjam[i] == 0):
12              # Cek umur termuda
13              if umur[i] < umur_min:
14                  umur_min = umur[i]
15                  kandidat = i
16          i += 1
17      return kandidat
18  # program utama
19  # input
20  N = int(input("Masukkan jumlah orang: "))
21  matrix = []
22  umur = [0 for _ in range(N)]
23  sudah_pinjam = [0 for _ in range(N)]
24  riwayat_pinjam = []
25  for i in range(N):
26      col = [0]*N
27      matrix += [col]
28
29  for y in range(N):
30      for x in range(N):
31          val = int(input(f'Masukkan isi matrix
              ketetanggaan untuk koordinat ({x},{y}): '))
```

```
32          matrix[y][x] = val
33      print('================================')
34
35  for i in range(N):
36      val_umur = int(input(f'Masukkan umur untuk index {i
            }: '))
37      umur[i] = val_umur
38
39  index_pinjam = int(input('Masukkan index tempat awal
        buku berada: '))
40  awal = time.perf_counter()
41  sudah_pinjam[index_pinjam] = 1
42  riwayat_pinjam += [index_pinjam]
43  top = 0
44  print('Urutan index peminjaman buku: ', end='')
45  print(index_pinjam, end='')
46  while True:
47      koneksi = matrix[index_pinjam]
48      kandidat = cari_kandidat(koneksi, umur, sudah_pinjam
            , N)
49      if kandidat != -1:
50          index_pinjam = kandidat
51          sudah_pinjam[index_pinjam] = 1
52          riwayat_pinjam += [index_pinjam]
53          top += 1
54          print(" ->", index_pinjam, end="")
55      else:
56          # Pengecekan apakah semua orang sudah pernah
                meminjam
57          jumlah = 0
58          for items in sudah_pinjam:
59              jumlah += items
60          # selesai hanya jika semua orang sudah pernah
                meminjam dan buku kembali ke peminjam asal
61          if top == 0 and jumlah == N:
62              break
63          # Jika belum selesai tapi mentok, backtrack ke
                orang sebelumnya
64          top -= 1
65          index_pinjam = riwayat_pinjam[top]
66          print(" ->", index_pinjam, end="")
67
68  akhir = time.perf_counter()
```

```
eks = akhir - awal
print(f'\n\nWaktu eksekusi: {eks} detik.')
```

2.2.3 Algoritma Labirin

Kode 6: Kode Python algoritma Labirin dengan subprogram

```
import time
"""labirin (dengan Subprogram)"""
# definisi fungsi
def choose_dir(pointer):
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
        direction = [1,1,1,1]
    return direction
# program utama
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
```

```
34          val = int(input(f'Apakah koordinat ({x},{y})
                bisa dilewati (1) atau tidak (0): '))
35          matrix[y][x] = val
36      print('================================')
37
38  # proses
39  start_time = time.perf_counter()
40  # inisialisasi pointer dan arahnya
41  status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
42  # cari titik awal di sisi atas
43  for i in range(N):
44      if matrix[0][i] == 1:
45          status_sisi[0] = [i,0]
46          break
47  # cari titik awal di sisi bawah
48  for i in range(N):
49      if matrix[N-1][i] == 1:
50          status_sisi[1] = [i,N-1]
51          break
52  # cari titik awal di sisi kiri
53  for i in range(N):
54      if matrix[i][0] == 1:
55          status_sisi[2] = [0,i]
56          break
57  # cari titik awal di sisi kanan
58  for i in range(N):
59      if matrix[i][N-1] == 1:
60          status_sisi[3] = [N-1, i]
61          break
62
63  # tentukan endpoint
64  endpoints = []
65  for i in range(4):
66      if status_sisi[i] != [-1,-1]:
67          endpoints += [status_sisi[i]]
68
69  # tentukan titik mulai
70  pointer = endpoints[0]
71  direction = [0,0,0,0]
72  avail_move = [0,0,0,0]
73  forbidden_move = [0,0,0,0]
74  clockwise = True
75  change_clockwise = 0
```

```
76 # jalankan iterasi
77 while pointer != endpoints[1]:
78     change_clockwise += 1
79     print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
80     # cek arah yang tersedia
81     direction = choose_dir(pointer)
82     # cek apakah bisa dilewati
83     i = 0
84     while(i < 4):
85         if direction[i] == 1:
86             if (i == 0) and (forbidden_move[i] == 0):
87                 bukan_dinding = int(matrix[pointer
                   [1]-1][pointer[0]] == 1)
88                 avail_move[i] = bukan_dinding
89             elif (i == 1) and (forbidden_move[i] == 0):
90                 bukan_dinding = int(matrix[pointer[1]][
                   pointer[0]+1] == 1)
91                 avail_move[i] = bukan_dinding
92             elif (i == 2) and (forbidden_move[i] == 0):
93                 bukan_dinding = int(matrix[pointer
                   [1]+1][pointer[0]] == 1)
94                 avail_move[i] = bukan_dinding
95             elif (i == 3) and (forbidden_move[i] == 0):
96                 bukan_dinding = int(matrix[pointer[1]][
                   pointer[0]-1] == 1)
97                 avail_move[i] = bukan_dinding
98         else:
99             avail_move[i] = 0
100        i += 1
101    # Jika jalan buntu
102    if avail_move == [0,0,0,0]:
103        i = 0
104        while(i < 4):
105            if direction[i] == 1:
106                if i == 0:
107                    bukan_dinding = int(matrix[pointer
                       [1]-1][pointer[0]] == 1)
108                    avail_move[i] = bukan_dinding
109                elif i == 1:
110                    bukan_dinding = int(matrix[pointer
                       [1]][pointer[0]+1] == 1)
111                    avail_move[i] = bukan_dinding
```

```
112                 elif i == 2:
113                     bukan_dinding = int(matrix[pointer
                            [1]+1][pointer[0]] == 1)
114                     avail_move[i] = bukan_dinding
115                 else:
116                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]-1] == 1)
117                     avail_move[i] = bukan_dinding
118             else:
119                 avail_move[i] = 0
120             i += 1
121
122     # Pilih jalur
123     if clockwise:
124         j = 0
125         while (j < 4):
126             if avail_move[j] == 1:
127                 if j == 0:
128                     pointer[1] -= 1
129                     forbidden_move = [0,0,1,0]
130                     break
131                 elif j == 1:
132                     pointer[0] += 1
133                     forbidden_move = [0,0,0,1]
134                     break
135                 elif j == 2:
136                     pointer[1] += 1
137                     forbidden_move = [1,0,0,0]
138                     break
139                 else:
140                     pointer[0] -= 1
141                     forbidden_move = [0,1,0,0]
142                     break
143             j += 1
144     else:
145         j = 3
146         while (j >= 0):
147             if avail_move[j] == 1:
148                 if j == 0:
149                     pointer[1] -= 1
150                     forbidden_move = [0,0,1,0]
151                     break
152                 elif j == 1:
```

```
153                         pointer[0] += 1
154                         forbidden_move = [0,0,0,1]
155                         break
156                     elif j == 2:
157                         pointer[1] += 1
158                         forbidden_move = [1,0,0,0]
159                         break
160                     else:
161                         pointer[0] -= 1
162                         forbidden_move = [0,1,0,0]
163                         break
164                 j -= 1
165 
166     if (change_clockwise % (4*N)) == 0:
167         clockwise = not clockwise
168 
169 print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
    dilewati')
170 end_time = time.perf_counter()
171 execution_time = end_time - start_time
172 print('Labirin Diselesaikan!')
173 print(f"Waktu eksekusi: {execution_time:.6f} detik")
```